Machine Intelligence 1 Tutorial

07 Class Imbalance and RBF-Networks

Fachgebiet Neuronale Informationsverarbeitung (NI)

WiSe 25/26

Dealing with imbalanced data

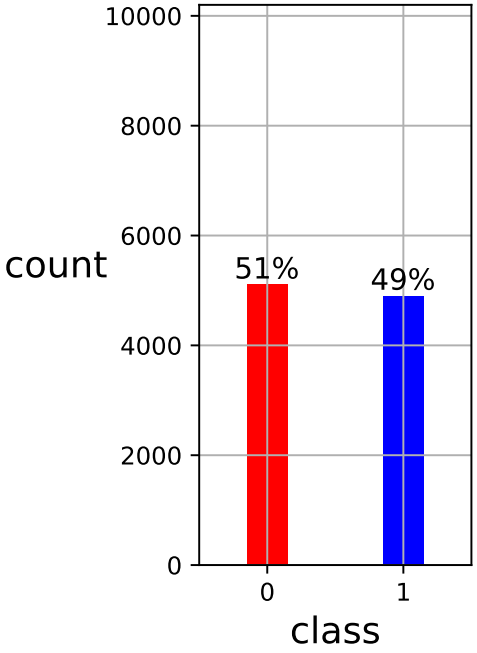
Fairness in MI

Non-parametric classification

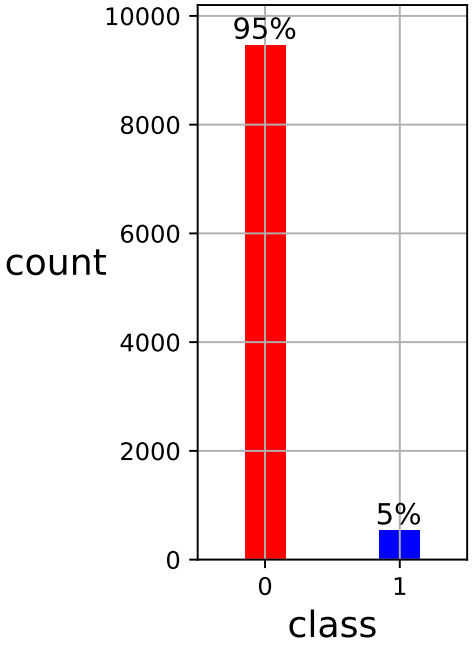
RBF-Networks

Motivation

Proportion of class labels in the dataset is not uniform. Not the same as "easy vs. difficult" classes. This is only about class frequency.

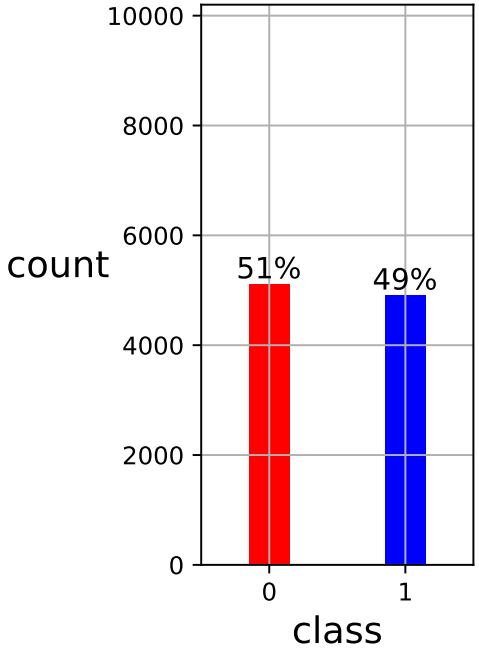


(a) balanced $\approx 1:1$

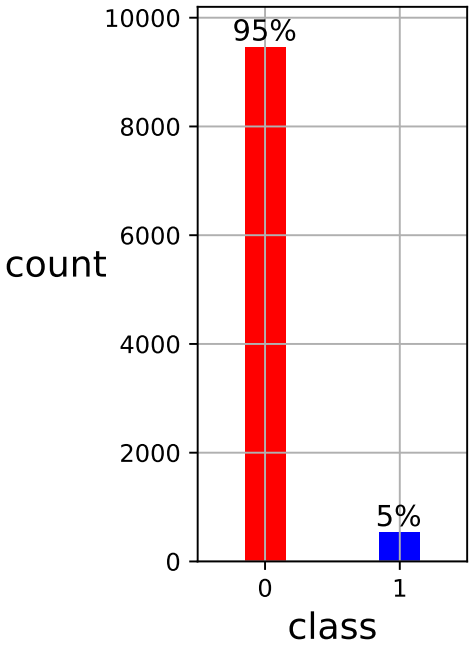


(b) highly unbalanced

Motivation



(a) balanced $\approx 1:1$

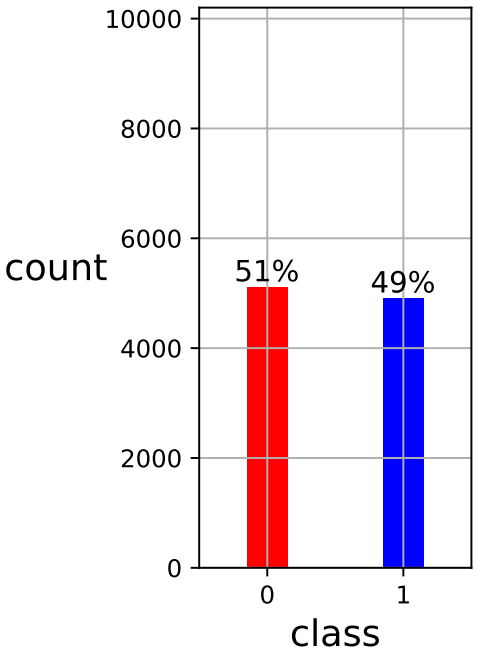


(b) highly unbalanced

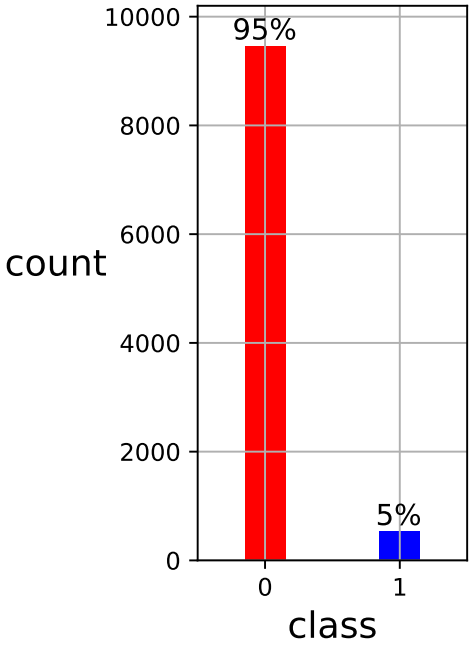
A strong imbalance leads to the classifier learning a trivial solution that indeed minimizes the average cost over the training samples.

$$E^T = \frac{1}{p} \sum_{\alpha=1}^{p} e^{(\alpha)}$$

Motivation



(a) balanced $\approx 1:1$



(b) highly unbalanced

$$E^T = \frac{1}{p}\Big[\sum_{\beta \in D_+}^{|D_+|} e^{(\beta)} + \sum_{\beta \in D_-}^{|D_-|} e^{(\beta)}\Big]$$

By always predicting "0" it will be correct 95% of the time.

Confusion matrix

The confusion matrix differentiates between the types of mistakes a classifier makes.

		Ground truth label	
		Positive	Negative
		"Cat"	"not Cat"
Prediction	Positive	**T**rue **P**ositives	**F**alse **P**ositives
	Negative	**F**alse **N**egatives	**T**rue **N**egatives
		P	N

Performance metrics

		Ground truth label	
		Positive	Negative
		"Cat"	"not Cat"
Prediction	Positive	**T**rue **P**ositives	**F**alse **P**ositives
	Negative	**F**alse **N**egatives	**T**rue **N**egatives
		P	N

fraction of correct classifications $\widehat{=}$ Accuracy $= \dfrac{TP + TN}{P + N}$

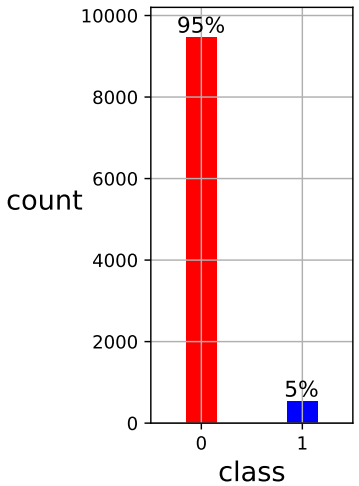
Performance metrics

		Ground truth label	
		Positive	Negative
		"Cat"	"not Cat"
Prediction	Positive	**T**rue **P**ositives	**F**alse **P**ositives
	Negative	**F**alse **N**egatives	**T**rue **N**egatives
		P	N

$$\text{fraction of correct classifications} \mathrel{\widehat{=}} \text{Accuracy} = \frac{TP + TN}{P + N}$$

In our imbalanced example:

$$\frac{TP + TN}{P + N} = \frac{0 + 9500}{500 + 9500} = 0.95$$



Performance metrics

		Ground truth label	
		Positive	Negative
		"Cat"	"not Cat"
Prediction	Positive	**T**rue **P**ositives	**F**alse **P**ositives
	Negative	**F**alse **N**egatives	**T**rue **N**egatives
		P	N

$$\text{fraction of correct classifications} \mathrel{\widehat{=}} \text{Accuracy} = \frac{TP + TN}{P + N}$$

Other metrics:
How many of the positive labels did we get:

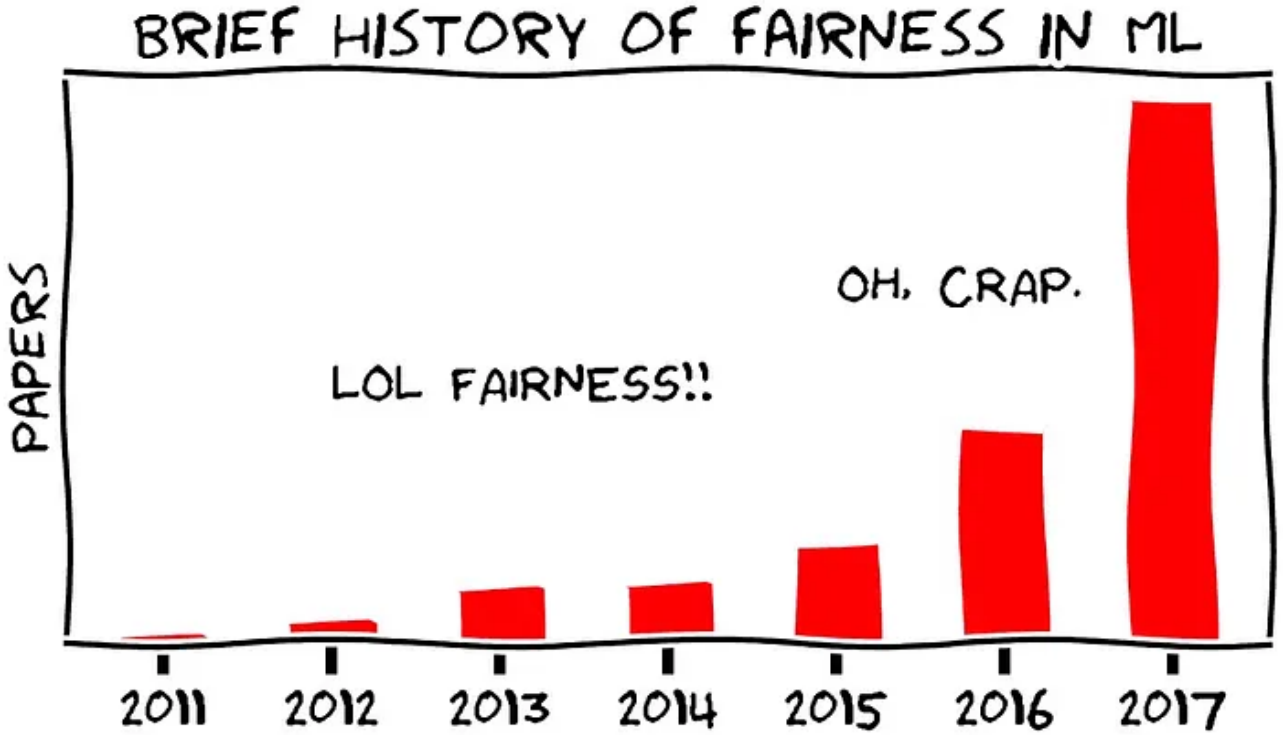
$$\text{sensitivity (recall)} = \frac{TP}{P}$$

Similarly, for the negative class. How many of the negative class did we get:

$$\text{specificity} = \frac{TN}{N}$$

Short Excursion

Aspects of Fairness in MI



[1]

[1] image taken from towardsdatascience.com

Aspects - What is "fair"?

1. legal aspects (e.g. to help identify discriminating features)
2. data representational of truth
3. ensure, data samples aren't
   1. *skewed*, i.e. future observations confirm prediction ("confirmation bias")
   2. *tainted*, i.e. bias existing in old data is kept
   3. *limited*, i.e. less reliable or fewer data samples of minority groups
   4. *proxies* i.e. delete sensible features leaves "proxies"/"hidden features" that MI learns

Aspects - What is "fair"?

1. legal aspects (e.g. to help identify discriminating features)
2. data representational of truth
3. ensure, data samples aren't
   1. *skewed*, i.e. future observations confirm prediction ("confirmation bias")
      e.g. less crime reported → less police → less crime detected → less crime reported
   2. *tainted*, i.e. bias existing in old data is kept
   3. *limited*, i.e. less reliable or fewer data samples of minority groups
   4. *proxies* i.e. delete sensible features leaves "proxies"/"hidden features" that MI learns

Aspects - What is "fair"?

1. legal aspects (e.g. to help identify discriminating features)
2. data representational of truth
3. ensure, data samples aren't
   1. *skewed*, i.e. future observations confirm prediction ("confirmation bias")
      e.g. less crime reported → less police → less crime detected → less crime reported
   2. *tainted*, i.e. bias existing in old data is kept
      e.g. less female/non-binary professors in the past → MI decides for new position in favour of male candidates
   3. *limited*, i.e. less reliable or fewer data samples of minority groups
   4. *proxies* i.e. delete sensible features leaves "proxies"/"hidden features" that MI learns

Aspects - What is "fair"?

1. legal aspects (e.g. to help identify discriminating features)
2. data representational of truth
3. ensure, data samples aren't
   1. *skewed*, i.e. future observations confirm prediction ("confirmation bias")
      e.g. less crime reported → less police → less crime detected → less crime reported
   2. *tainted*, i.e. bias existing in old data is kept
      e.g. less female/non-binary professors in the past → MI decides for new position in favour of male candidates
   3. *limited*, i.e. less reliable or fewer data samples of minority groups *e.g.* carcinoma detection on skin → less accuracy in minority group
   4. *proxies* i.e. delete sensible features leaves "proxies"/"hidden features" that MI learns

Aspects - What is "fair"?

1. legal aspects (e.g. to help identify discriminating features)
2. data representational of truth
3. ensure, data samples aren't
   1. *skewed*, i.e. future observations confirm prediction ("confirmation bias")
      e.g. less crime reported → less police → less crime detected → less crime reported
   2. *tainted*, i.e. bias existing in old data is kept
      e.g. less female/non-binary professors in the past → MI decides for new position in favour of male candidates
   3. *limited*, i.e. less reliable or fewer data samples of minority groups *e.g.* carcinoma detection on skin → less accuracy in minority group
   4. *proxies* i.e. delete sensible features leaves "proxies"/"hidden features" that MI learns *e.g.* exclude gender in CVs, parental leave duration remains

Include aspects of fairness - where and how?

(a) Model construction[2]
(b) Preprocessing[3]
(c) Optimization[4]

[2] e.g. see Learning Fair Representations by Zemel et al.
[3] e.g. see iFair: Learning Individually Fair Data Representations for Algorithmic Decision Making, by Lahoti et al.
[4] e.g. see Fairness Beyond Disparate Treatment & Disparate Impact: Learning Classification without Disparate Mistreatment, by Zafar et al.

Fairness via Model Construction - Example[5]

Setting: Binary classification - hiring for a job, one single sensitive attribute

- $\{\underline{\mathbf{x}}^{(\alpha)} \in \mathbb{R}^N\}_{\alpha=1}^p$; e.g. entries for education, experience, grades, ...
- $y_T^{(\alpha)} \in 0, 1$; e.g. hire/no-hire
- $\{a^{(\alpha)} \in \{0, 1\}\}_{\alpha=1}^p$; sensitive attribute per applicant
- $C(\underline{\mathbf{x}}, a)$ as binary predictor.

[5] from towardsdatascience.com

Fairness via Model Construction - Approaches

a. **Unawareness**: exclude sensitive attribute in training data
b. **Demographic parity**:

c. **Equalized odds**:

Fairness via Model Construction - Approaches

a. **Unawareness**: exclude sensitive attribute in training data

b. **Demographic parity**: ensure that probabilities of (non-)hiring are the same for both groups

$$P_{a=0}(C(\underline{\mathbf{x}}, a) = y) = P_{a=1}(C(\underline{\mathbf{x}}, a) = y),\ y \in \{0, 1\}$$

c. **Equalized odds**:

Fairness via Model Construction - Approaches

a. **Unawareness**: exclude sensitive attribute in training data

b. **Demographic parity**: ensure that probabilities of (non-)hiring are the same for both groups

$$P_{a=0}(C(\underline{\mathbf{x}}, a) = y) = P_{a=1}(C(\underline{\mathbf{x}}, a) = y),\ y \in \{0, 1\}$$

c. **Equalized odds**: ensure same conditional probabilities

$$P_{a=0}(C = y | y_T = \hat{y}) = P_{a=1}(C = y | y_T = \hat{y}),\ \forall y, \hat{y} \in \{0, 1\}$$

From Non-parametric classification to RBF-Networks

The setting for M-way classification

Data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{y}}_T^{(\alpha)} \right) \right\}_{\alpha=1}^{p}$$

where $\underline{\mathbf{y}}_T^{(\alpha)} \in \{0,1\}^M$ with $\sum_{c=1}^{M} (y_T)_c = 1$ (one-hot encoding of class labels)

The setting for M-way classification

Data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{y}}_T^{(\alpha)} \right) \right\}_{\alpha=1}^{p}$$

where $\underline{\mathbf{y}}_T^{(\alpha)} \in \{0,1\}^M$ with $\sum_{c=1}^{M} (y_T)_c = 1$ (one-hot encoding of class labels)

Model:

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) \in \mathbb{R}^M$$

with $\sum_{c=1}^{M} y_c(\underline{\mathbf{x}}) = 1$ and $y_c(\underline{\mathbf{x}}) \geq 0 \, \forall \, c$

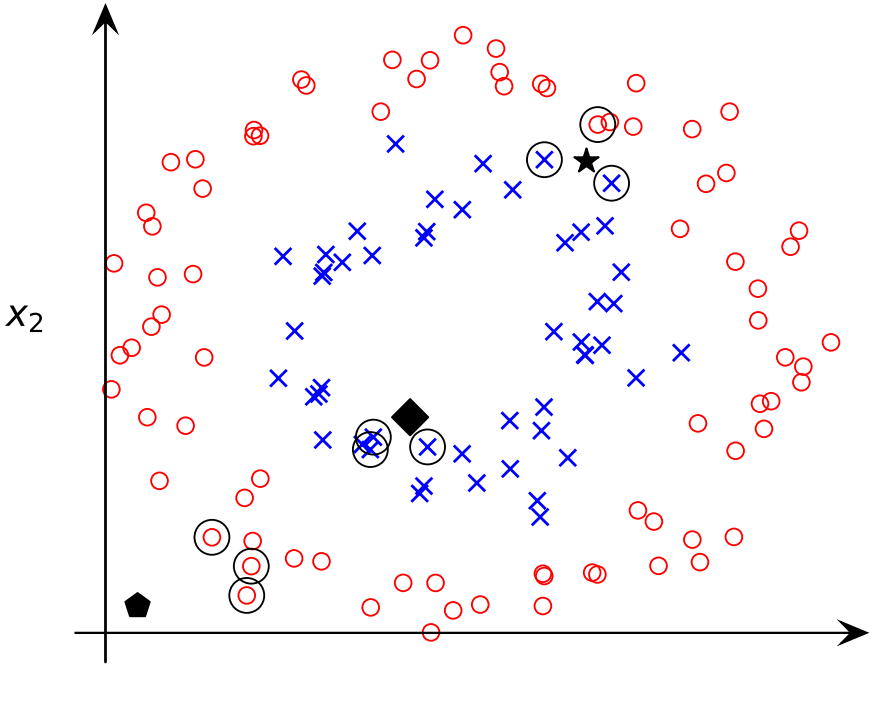
Predictions $\underline{\mathbf{y}}(\underline{\mathbf{x}})$ can be used to give

- probabilities of the predicted class (e.g. $y_5(\underline{\mathbf{x}}) = 0.75 \rightsquigarrow$ class "5" with 75% probability.
- hard decisions:

$$\underset{c=1,\ldots,M}{\operatorname{argmax}} \; y_c(\underline{\mathbf{x}})$$

k nearest neighbor classifier (kNN)

Prediction follows the majority vote of the k nearest neighbors around a query point.



Let kNN($\underline{\mathbf{x}}$) be the indices $\{\beta_1, \beta_2, \ldots, \beta_k\}$ of the k data points closest to $\underline{\mathbf{x}}$ w.r.t. Euclidean norm:

$$\beta_j = \underset{\substack{\alpha \in \{1,\ldots,p\} \setminus \\ \{\beta_1,\ldots,\beta_{j-1}\}}}{\operatorname{argmin}} \|\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}\|_2$$

For multi-class classification

For multi-class classification the kNN classifier (for fixed $k \mathrel{\widehat{=}}$ hyper-parameter) is defined by:

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) = \frac{1}{k} \sum_{\beta \in k\mathrm{NN}(\underline{\mathbf{x}})} \underline{\mathbf{y}}_T^{(\beta)}$$

$\underline{\mathbf{y}}(\underline{\mathbf{x}})$ is effectively an arithmetic average of the labels in the neighborhood.

Example $k = 3$, $M = 4$:

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) = \frac{1}{3}\left[\begin{pmatrix}1\\0\\0\\0\end{pmatrix} + \begin{pmatrix}1\\0\\0\\0\end{pmatrix} + \begin{pmatrix}0\\1\\0\\0\end{pmatrix}\right] = \frac{1}{3}\begin{pmatrix}2\\1\\0\\0\end{pmatrix} = \begin{pmatrix}2/3\\1/3\\0\\0\end{pmatrix}$$

applying a hard decision would yield $c = 1$.

the binary classification case

For binary classification, we can set $M = 2$ for a multi-class kNN classifier. Alternatively, we can construct the kNN classifier using *scalar* labels:

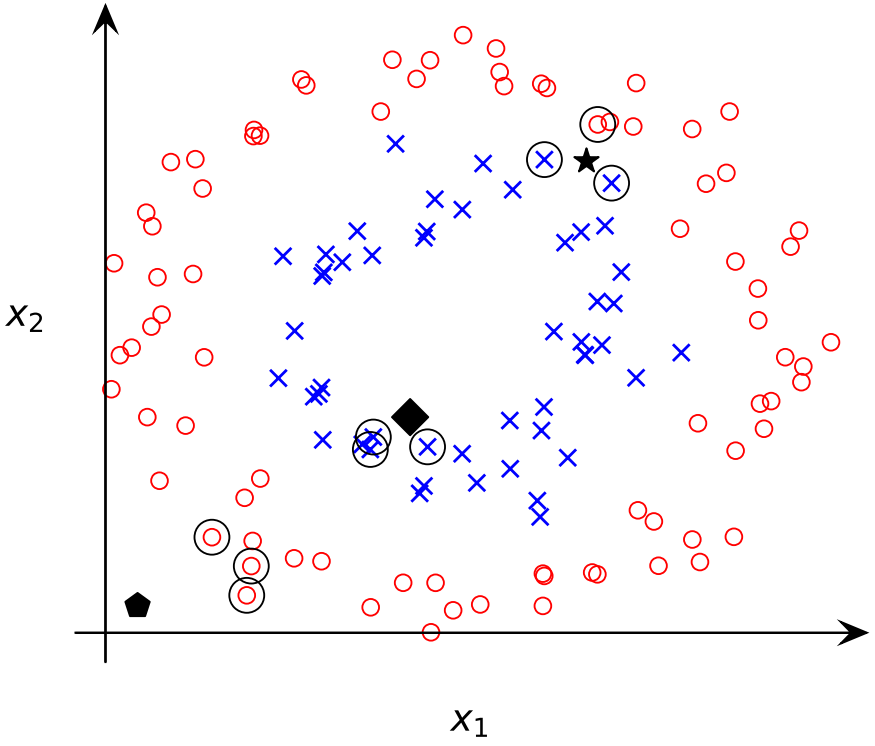
$$y_T \in \{0, 1\} \quad \text{and} \quad y(\underline{\mathbf{x}}) \in \mathbb{R}$$

with

$$y(\underline{\mathbf{x}}) \geq 0 : \begin{cases} y(\underline{\mathbf{x}}) & \text{probability of class } 1 \\ 1 - y(\underline{\mathbf{x}}) & \text{probability of class } 0 \end{cases}$$

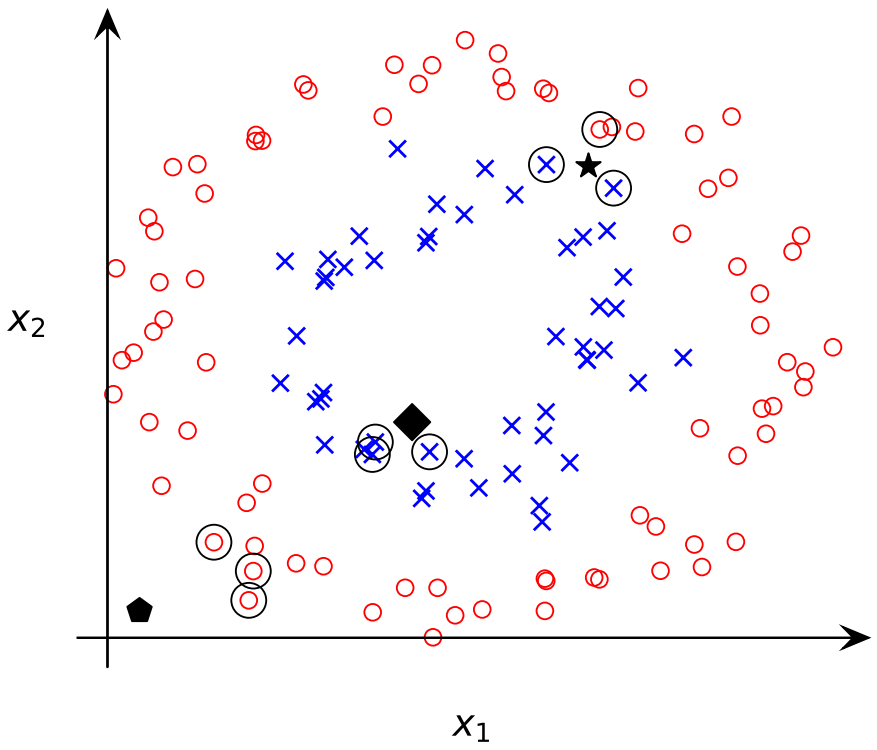
Q: *What is the role of k in terms of model complexity?* $k = 1$ *vs.* $k = p$

Hint: Jitter the data (or the query points, but not both) and see which choice of k leads to different (more noisy) predictions.



Q: *What is the role of k in terms of model complexity?* $k = 1$ *vs.* $k = p$

Hint: Jitter the data (or the query points, but not both) and see which choice of k leads to different (more noisy) predictions.



Q: *How would you programatically find the k nearest neighbors for some query point $\underline{\mathbf{x}}$?*

Parzen window classification

kNN with all points, but weighted by a window.

Parzen window classification

Prediction is based on

- all data points
- weighted according to some window/"kernel" function $\kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}')$.

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) := \frac{1}{Z} \sum_{\alpha=1}^{p} \underline{\mathbf{y}}_T^{(\alpha)} \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)})$$

with

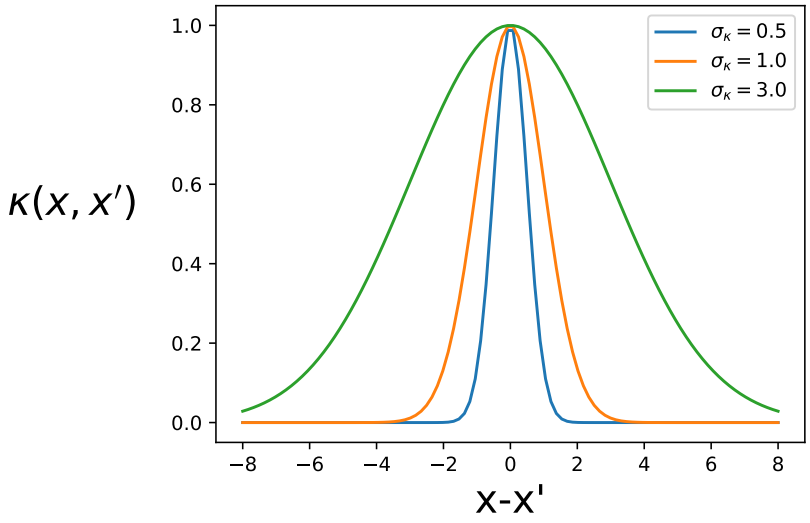
$$Z := \sum_{\alpha=1}^{p} \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)})$$

The Gaussian kernel

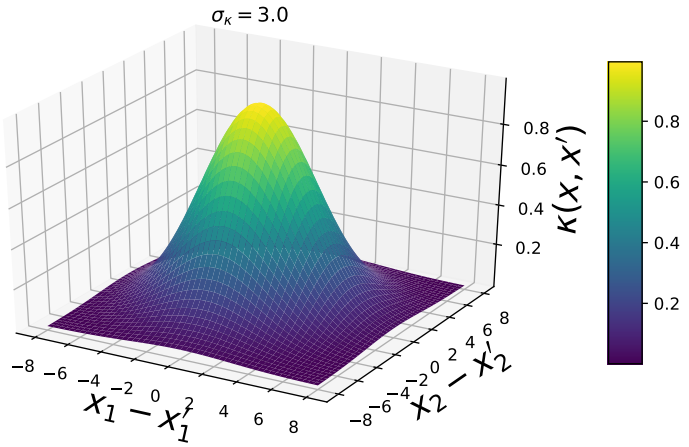
An example for the window/"kernel" function $\kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}')$ would be the Gaussian function:

$$\kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}') := \exp\left(-\frac{\|\underline{\mathbf{x}} - \underline{\mathbf{x}}'\|_2^2}{2\,\sigma_\kappa^2}\right)$$

where σ_κ is referred to as the *width* of the kernel.

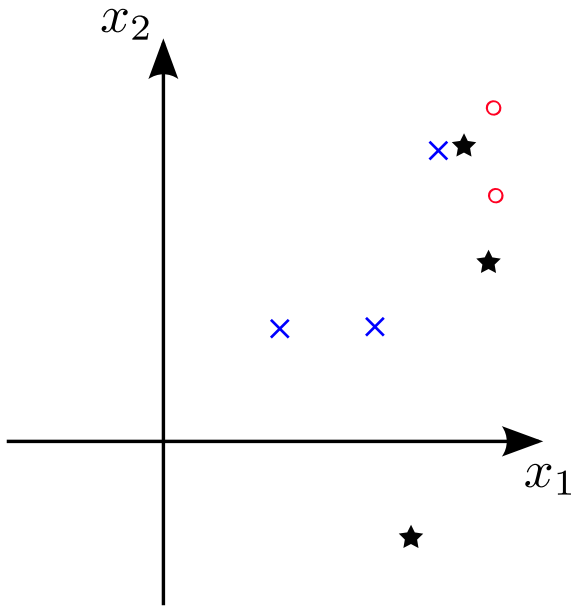


(a) For data in 1D



(b) For data in 2D

Binary classification example with 2D data

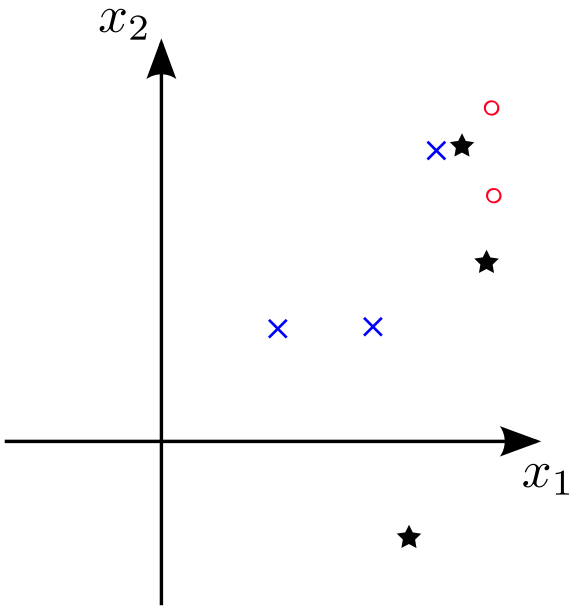


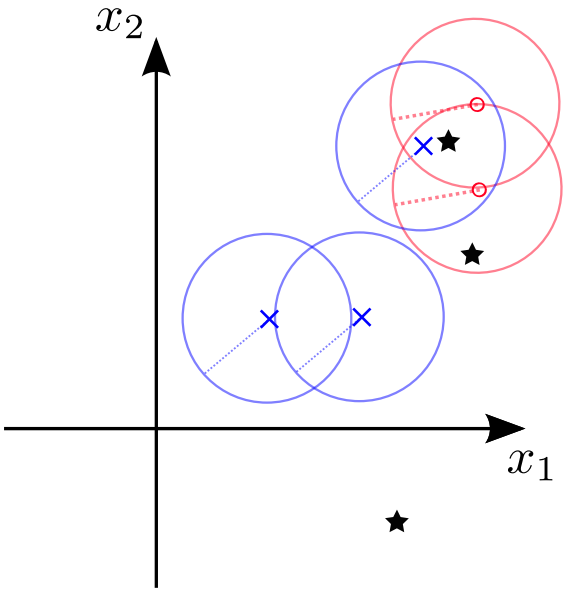
(a) Binary classification data in 2D

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) := \frac{1}{Z} \sum_{\alpha=1}^{p} \underline{\mathbf{y}}_T^{(\alpha)} \, \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)})$$

$$\kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)}) := \exp\left(- \frac{\|\underline{\mathbf{x}} - \underline{\mathbf{x}}^{(\alpha)}\|_2^2}{2\,\sigma_\kappa^2} \right)$$

Binary classification example with 2D data

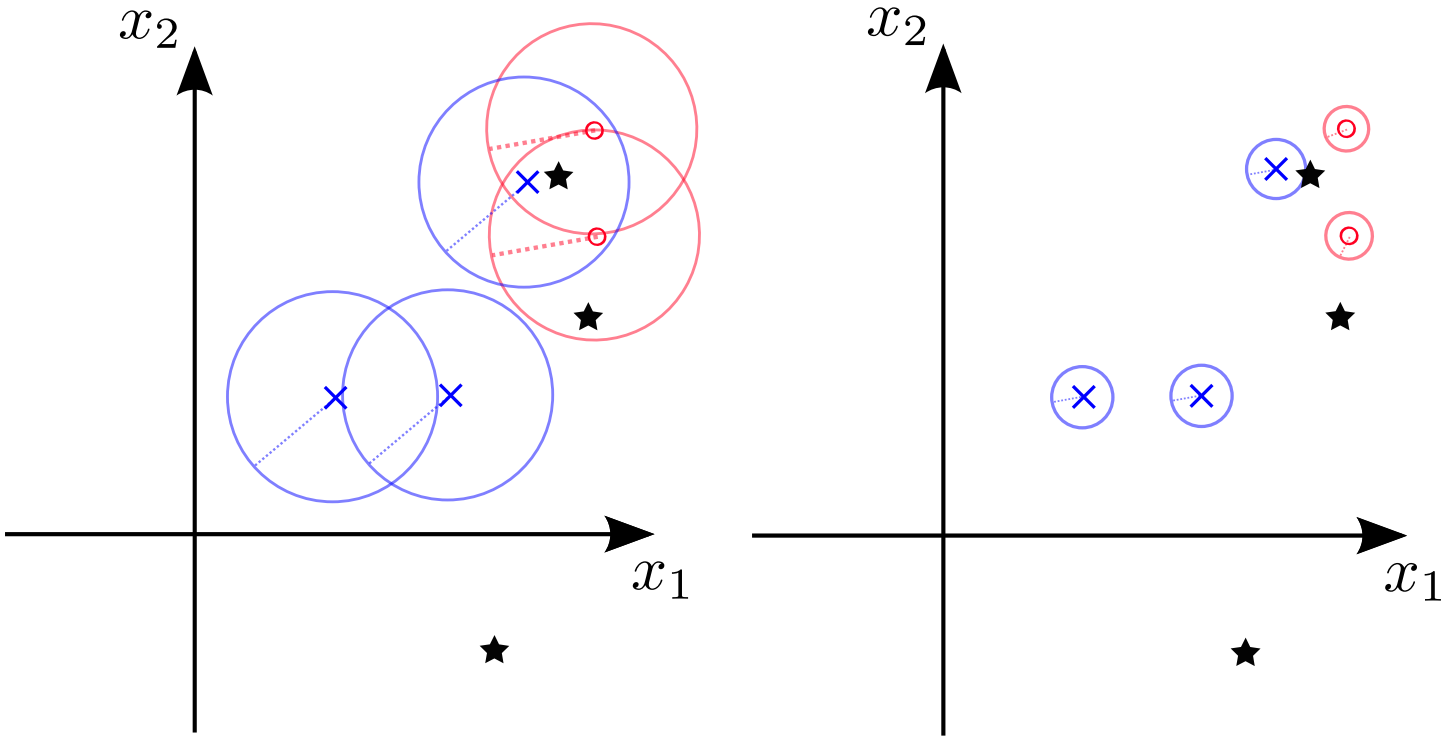




(a) Binary classification data in 2D

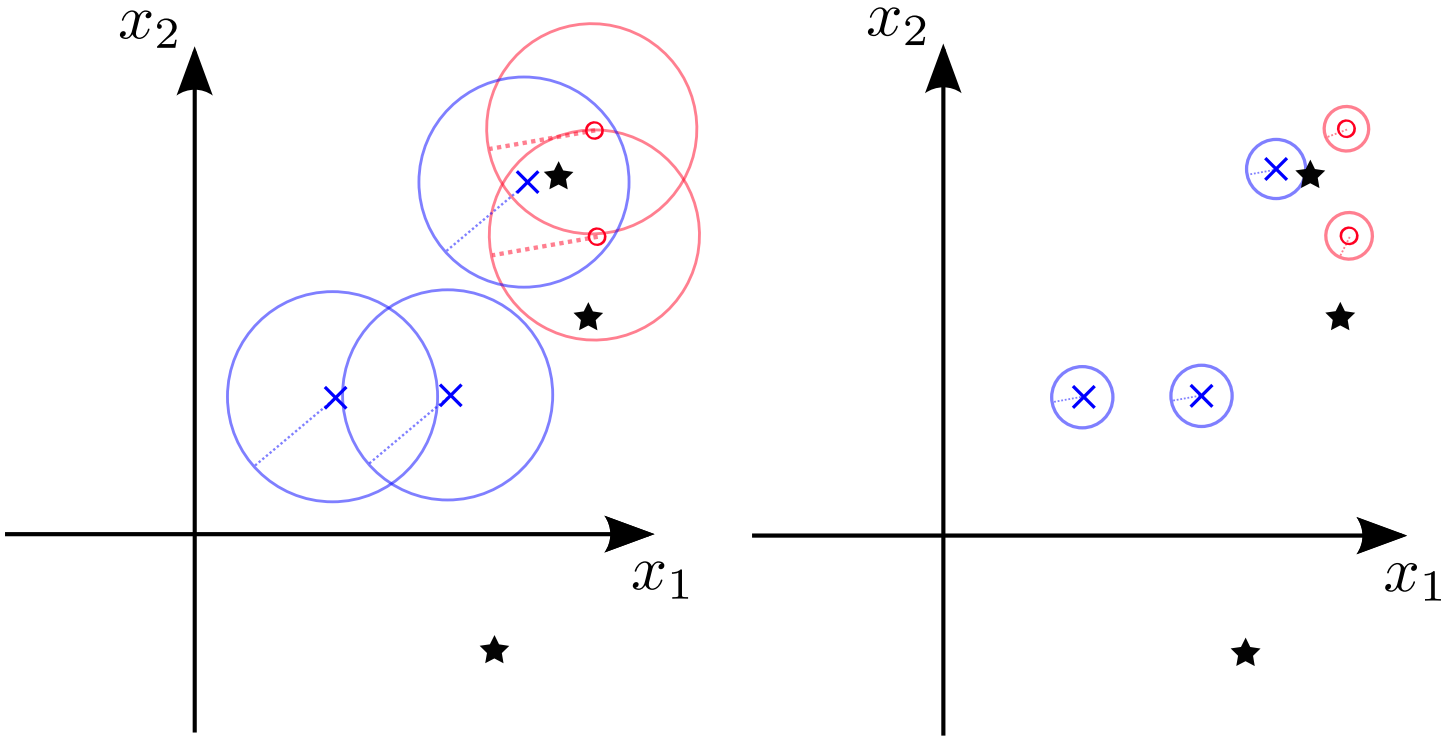
$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) := \frac{1}{Z} \sum_{\alpha=1}^{p} \underline{\mathbf{y}}_T^{(\alpha)} \, \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)})$$

$$\kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)}) := \exp\left(- \frac{\|\underline{\mathbf{x}} - \underline{\mathbf{x}}^{(\alpha)}\|_2^2}{2\,\sigma_\kappa^2} \right)$$



(a) wide kernel width (b) narrow kernel width

Q: *What is the role of the kernel width σ_κ in terms of model complexity?*



(a) wide kernel width (b) narrow kernel width

Q: *What is the role of the kernel width σ_κ in terms of model complexity?*

Hint: Jitter the query points (or the data) and see which choice of σ_κ will lead to different (more noisy) predictions.

Remarks:

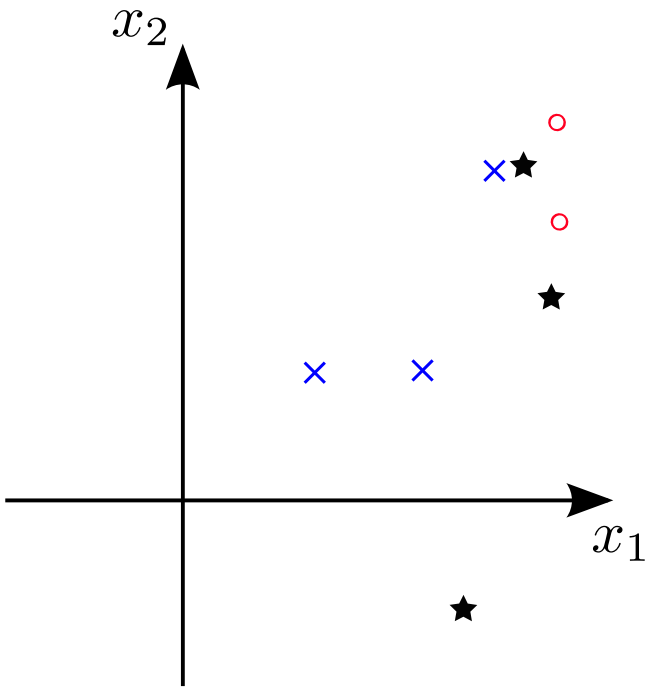
1. The width of the kernel κ controls the model complexity (narrow width $\rightsquigarrow$ over-fitting (high variance), wide $\rightsquigarrow$ under-fitting)
   One way to go about this is to apply a small noise to the query points (i.e. jitter) and see how the predictions will vary with different σ_κ. If this small noise in the position of the points leads to different predictions we are basically looking at high variance.
2. Using all data points. Potentially costly with large p

RBF-Networks

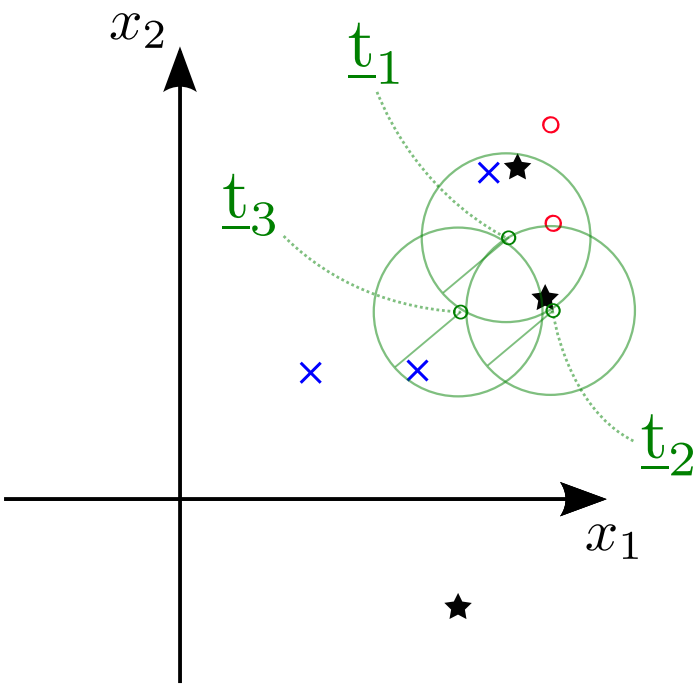
Parzen window classification with “representatives”

RBF-Networks

A trade-off between kNN classification and Parzen window classification.
Instead of taking all p points to predict the class probabilities (Parzen), limit the prediction to $k \ll p$ ~~points~~ "representative" functions.
Represent all points $\underline{\mathbf{x}}$ in terms of basis functions $\phi_i(\underline{\mathbf{x}}), i = 1 \ldots, k$
Let k be the number of basis functions.



(a) Two-class data in 2D



(b) $k = 3$ "representatives"

Nonlinear basis function

Feature transformation ($\underline{\mathbf{x}} \in \mathbb{R}^N$):

$$\underline{\phi} : \underline{\mathbf{x}} \mapsto \underline{\phi}(\underline{\mathbf{x}})$$

Assume $\{\underline{\mathbf{t}}_i\}$ and corresponding kernel width $\{\sigma_i\}$ have already been determined.
Transform the feature space using *radial basis functions* (Gaussian kernel functions):

$$\phi_i(\underline{\mathbf{x}}) \in \exp\left(-\frac{\|\underline{\mathbf{x}} - \underline{\mathbf{t}}_i\|_2^2}{2\,\sigma_i^2}\right), \quad i = 1, \ldots, k$$

Setting for binary classification/ scalar regression

Data:

transformed input $\underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \big(\underbrace{1}_{\phi_0}, \phi_1(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_k(\underline{\mathbf{x}}^{(\alpha)})\big)^\top$

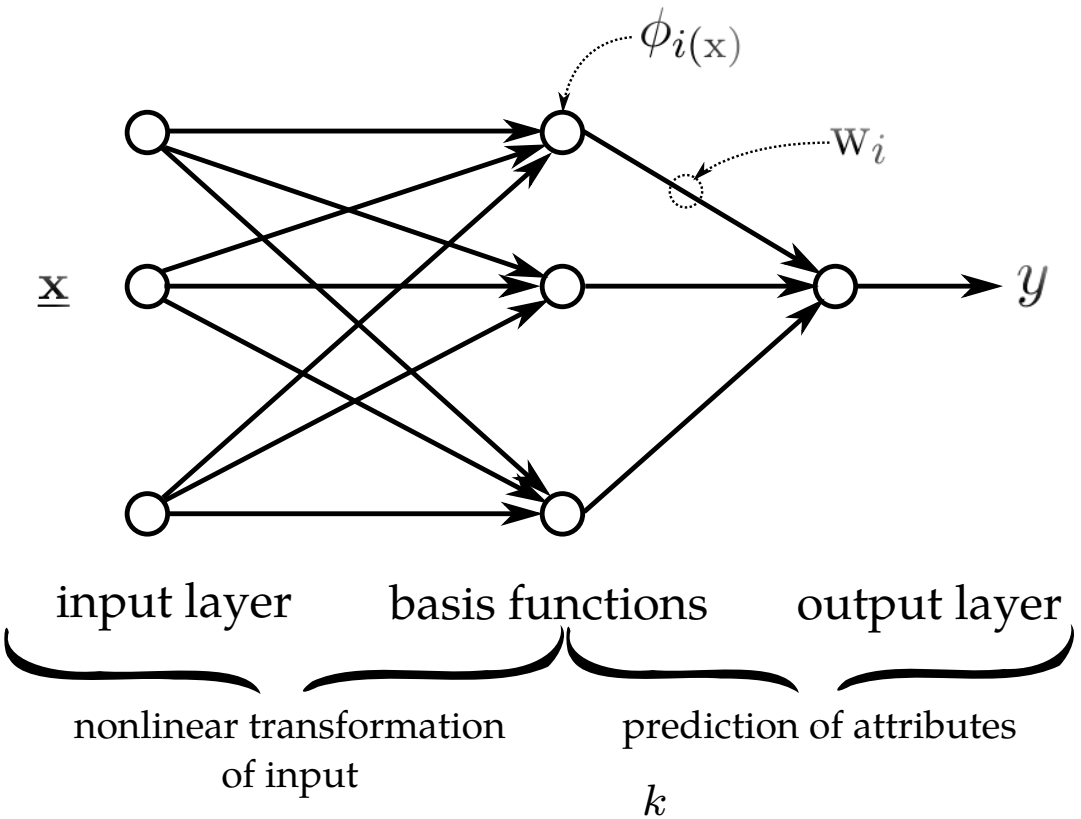
$\Rightarrow \underline{\phi}^{(\alpha)} \in \mathbb{R}^{k+1}$

label $y_T^{(\alpha)} \in \mathbb{R}$

with $\alpha = 1, \ldots, p$

Setting for binary classification/ scalar regression

Model:



$$= \sum_i \mathrm{w}_i \phi_i(\underline{\mathbf{x}})$$

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \sum_{i=0}^{k} w_i \phi_i(\underline{\mathbf{x}}) = \underline{\mathbf{w}}^\top \underline{\phi}$$

Don't forget the bias! Set $\underline{\mathbf{\Phi}} = \left(\underline{\phi}^{(1)}, \ldots, \underline{\phi}^{(\alpha)}, \ldots, \underline{\phi}^{(p)}\right) \in \mathbb{R}^{k+1,p}$

quadratic cost function.

Setting for binary classification/ scalar regression

Model:

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \sum_{i=0}^{k} w_i \phi_i(\underline{\mathbf{x}}) = \underline{\mathbf{w}}^\top \underline{\phi}$$

Don't forget the bias! Set $\underline{\mathbf{\Phi}} = \left(\underline{\phi}^{(1)}, \ldots, \underline{\phi}^{(\alpha)}, \ldots, \underline{\phi}^{(p)}\right) \in \mathbb{R}^{k+1,p}$
quadratic cost function. Reuse solution for linear regression:

$$\underline{\mathbf{w}}^* = \left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top\right)^{-1} \underline{\mathbf{\Phi}}\,\underline{\mathbf{y}}_{\mathsf{True}}^\top$$

Q: *What about using RBF-Networks for classification?*

Setting for binary classification/ scalar regression

Model:

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \sum_{i=0}^{k} w_i \phi_i(\underline{\mathbf{x}}) = \underline{\mathbf{w}}^\top \underline{\phi}$$

Don't forget the bias! Set $\underline{\mathbf{\Phi}} = \left(\underline{\phi}^{(1)}, \ldots, \underline{\phi}^{(\alpha)}, \ldots, \underline{\phi}^{(p)}\right) \in \mathbb{R}^{k+1,p}$ quadratic cost function. Reuse solution for linear regression:

$$\underline{\mathbf{w}}^* = \left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top\right)^{-1} \underline{\mathbf{\Phi}}\,\underline{\mathbf{y}}_{\text{True}}^\top$$

Q: *What about using RBF-Networks for classification?*

- The quadratic cost function can still be applied to learning the parameters in an RBF-Network for solving classification problems as well but the performance is often less than when using cross-entropy loss.

Multi-dimensional RBF-Networks

Generalizing nonlinear basis functions in linear models:

$$\underline{\mathbf{y}}(\underline{\phi}(\underline{\mathbf{x}})) = \begin{pmatrix} y_1(\underline{\phi}(\underline{\mathbf{x}})) \\ y_2(\underline{\phi}(\underline{\mathbf{x}})) \\ \vdots \\ y_M(\underline{\phi}(\underline{\mathbf{x}})) \end{pmatrix} = \underline{\mathbf{W}}^\top \underline{\phi}(\underline{\mathbf{x}})$$

with $\underline{\mathbf{W}} \in \mathbb{R}^{k+1,M}$

closed form solution for quadratic cost:

$$\underline{\mathbf{W}}^* = \underbrace{\left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top\right)^{-1}}_{\substack{\text{assume} \\ \text{invertibility}}} \underline{\mathbf{\Phi}}\,\underline{\mathbf{Y}}_{\text{True}}^\top$$

Q: *What is $\underline{\mathbf{Y}}_{True}$? What is the shape of this matrix?*

Multi-dimensional RBF-Networks

Generalizing nonlinear basis functions in linear models:

$$\underline{\mathbf{y}}(\underline{\phi}(\underline{\mathbf{x}})) = \begin{pmatrix} y_1(\underline{\phi}(\underline{\mathbf{x}})) \\ y_2(\underline{\phi}(\underline{\mathbf{x}})) \\ \vdots \\ y_M(\underline{\phi}(\underline{\mathbf{x}})) \end{pmatrix} = \underline{\mathbf{W}}^\top \underline{\phi}(\underline{\mathbf{x}})$$

with $\underline{\mathbf{W}} \in \mathbb{R}^{k+1,M}$

closed form solution for quadratic cost:

$$\underline{\mathbf{W}}^* = \underbrace{\left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top\right)^{-1}}_{\substack{\text{assume} \\ \text{invertibility}}} \underline{\mathbf{\Phi}}\,\underline{\mathbf{Y}}_{\text{True}}^\top$$

Q: *What is $\underline{\mathbf{Y}}_{True}$? What is the shape of this matrix?*

$$\underline{\mathbf{Y}}_{\text{True}} := \left(\underline{\mathbf{y}}_{\text{True}}^{(1)}, \ldots, \underline{\mathbf{y}}_{\text{True}}^{(p)}\right) \in \mathbb{R}^{M,p}$$

Remarks:

- RBF-Networks are universal function approximators like MLPs.
- RBF-Networks suffer from the "curse of dimensionality"
- Regularization is needed:
  For weight decay:

$$\underline{\mathbf{W}}^* = \left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top + \lambda\,\underline{\mathbf{I}}_{k+1}\right)^{-1}\underline{\mathbf{\Phi}}\,\underline{\mathbf{Y}}_{\text{True}}^\top$$

RBF-Network vs. Parzen window classification

$$\underline{\mathbf{y}}_{\text{Parzen}}(\underline{\mathbf{x}}) := \frac{1}{Z} \sum_{\alpha=1}^{p} \underline{\mathbf{y}}_T^{(\alpha)} \, \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)}) \qquad \text{(Parzen window classifier)}$$

$$y_{\text{RBF-Net}}(\underline{\phi}; \underline{\mathbf{w}}) = \sum_{i=0}^{k} w_i \phi_i(\underline{\mathbf{x}}) \qquad \text{(RBF-Network)}$$

Q: *Which model class is a generalization of the other?*

RBF-Network vs. Parzen window classification

$$\underline{\mathbf{y}}_{\text{Parzen}}(\underline{\mathbf{x}}) := \frac{1}{Z}\sum_{\alpha=1}^{p} \underline{\mathbf{y}}_T^{(\alpha)}\, \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)}) \qquad \text{(Parzen window classifier)}$$

$$y_{\text{RBF-Net}}(\underline{\phi}; \underline{\mathbf{w}}) = \sum_{i=0}^{k} w_i \phi_i(\underline{\mathbf{x}}) \qquad \text{(RBF-Network)}$$

Q: *Which model class is a generalization of the other?*

Q: *How would you construct an RBF-Network to mimic a Parzen window classifier?*

RBF-Network vs. Parzen window classification

$$\underline{\mathbf{y}}_{\text{Parzen}}(\underline{\mathbf{x}}) := \frac{1}{Z}\sum_{\alpha=1}^{p} \underline{\mathbf{y}}_T^{(\alpha)}\, \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)}) \qquad \text{(Parzen window classifier)}$$

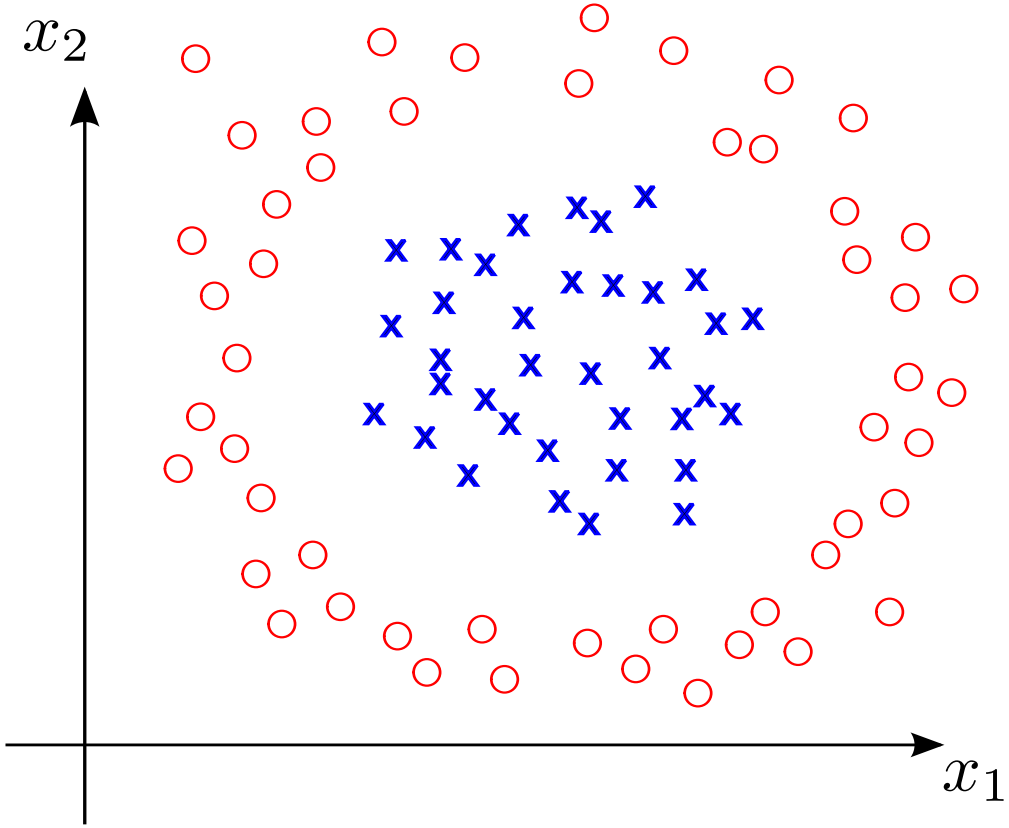
$$y_{\text{RBF-Net}}(\underline{\phi}; \underline{\mathbf{w}}) = \sum_{i=0}^{k} w_i \phi_i(\underline{\mathbf{x}}) \qquad \text{(RBF-Network)}$$

Q: *Which model class is a generalization of the other?*

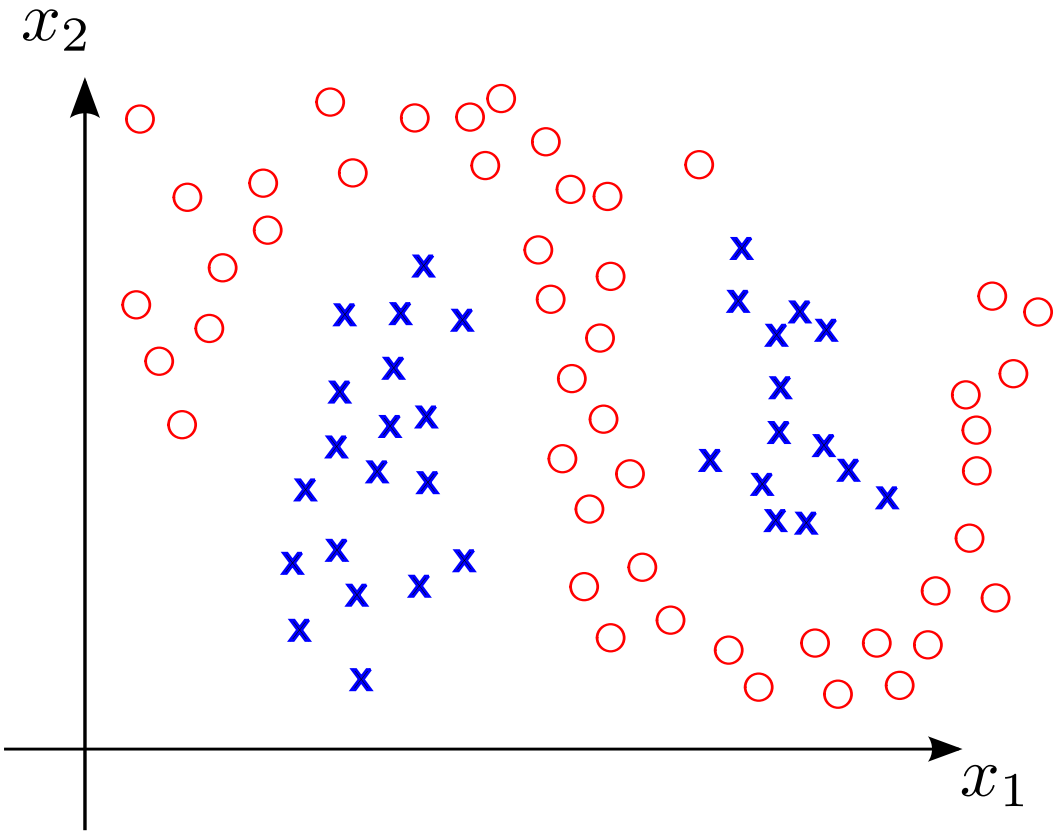
Q: *How would you construct an RBF-Network to mimic a Parzen window classifier?*

- An RBF-Network can be constructed and parameterized to act as a Parzen window classifier by placing a "representative'/centroid over every point $^{(\alpha)}$.

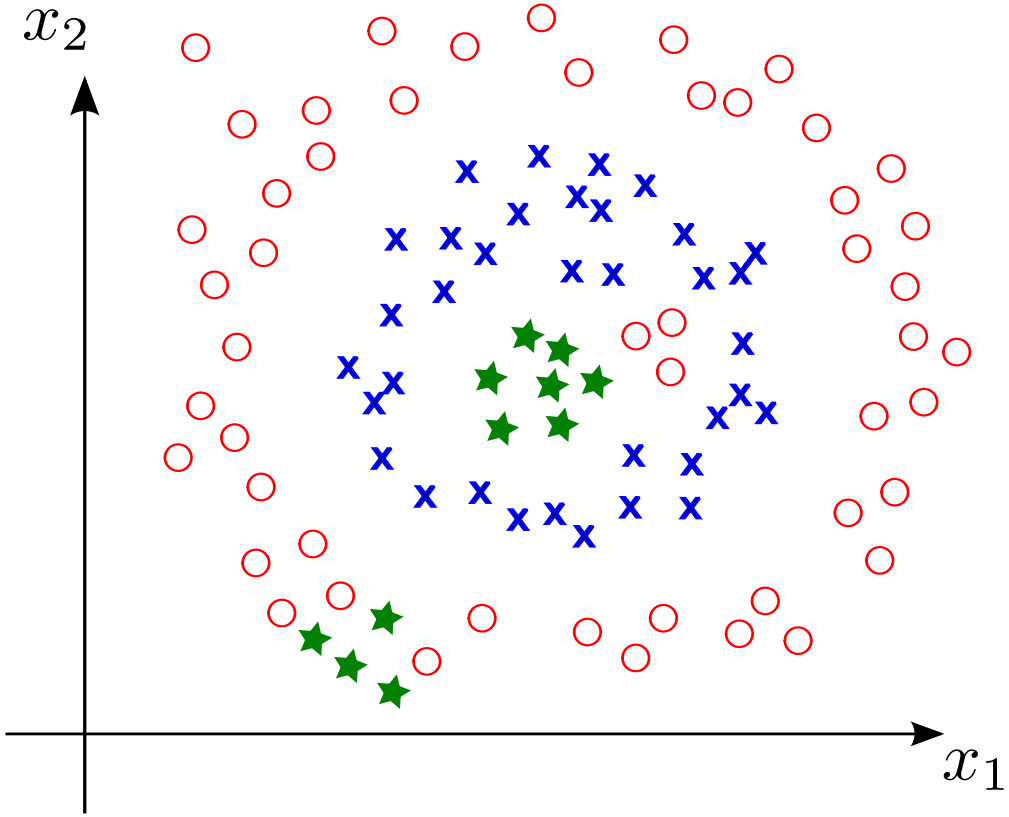
Q: *Can an RBF-Network solve this problem? How?*



Q: *Can an RBF-Network solve this problem? How many basis functions does it need?*



Q: *Can an RBF-Network solve this problem using only 2 centroids?*



MLPs	RBF-Networks

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters
+ usually better performance with larger datasets	+ suitable for small datasets

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters
+ usually better performance with larger datasets	+ suitable for small datasets
- data hungry	+ faster training with small # of basis functions

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters
+ usually better performance with larger datasets	+ suitable for small datasets
- data hungry	+ faster training with small # of basis functions
+ adapts functions to data and labels	can benefit from unlabeled data (semi-supervised learning)

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters
+ usually better performance with larger datasets	+ suitable for small datasets
- data hungry	+ faster training with small # of basis functions
+ adapts functions to data and labels	can benefit from unlabeled data (semi-supervised learning)
common neuronal mode between hidden and output neurons	The hidden layer serves a very different purpose than the output layer

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters
+ usually better performance with larger datasets	+ suitable for small datasets
- data hungry	+ faster training with small # of basis functions
+ adapts functions to data and labels	can benefit from unlabeled data (semi-supervised learning)
common neuronal mode between hidden and output neurons	The hidden layer serves a very different purpose than the output layer
total input of a hidden neuron determined via inner product $\underline{\mathbf{w}}_i^\top \underline{\mathbf{x}}$	total input is based on a Euclidean distance

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters
+ usually better performance with larger datasets	+ suitable for small datasets
- data hungry	+ faster training with small # of basis functions
+ adapts functions to data and labels	can benefit from unlabeled data (semi-supervised learning)
common neuronal mode between hidden and output neurons	The hidden layer serves a very different purpose than the output layer
total input of a hidden neuron determined via inner product $\underline{\mathbf{w}}_i^\top \underline{\mathbf{x}}$	total input is based on a Euclidean distance
	restricted function class (many basis functions needed for complicated approximations)

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters
+ usually better performance with larger datasets	+ suitable for small datasets
- data hungry	+ faster training with small # of basis functions
+ adapts functions to data and labels	can benefit from unlabeled data (semi-supervised learning)
common neuronal mode between hidden and output neurons	The hidden layer serves a very different purpose than the output layer
total input of a hidden neuron determined via inner product $\underline{\mathbf{w}}_i^\top \underline{\mathbf{x}}$	total input is based on a Euclidean distance
	restricted function class (many basis functions needed for complicated approximations)
	benefits from clustered structure in the data